PROYECTO FINAL INTEGRADOR: SISTEMA DISTRIBUIDO DE RESERVA DE BOLETERÍA CON ZEROC ICE & VANILLA JS (WEBPACK)

Arquitectura Híbrida RPC (TCP/WebSocket) | Concurrencia | Componentes Modulares

Código Asignatura:	09810 - TIC	**Período Académico:**	2026-2
Grupo / NRC:	NRC 12378		
Ponderación:	20.0 % (Proyecto Final del Curso)	**Política IAG:**	Nivel 3: Colaboración Asistida
Integrante:		**Código 1:**	

1 Contexto del Problema y Arquitectura del Sistema

Las plataformas de comercio electrónico de alta demanda, tales como las taquillas virtuales para conciertos masivos y eventos deportivos, representan uno de los mayores desafíos en la ingeniería de sistemas distribuidos. En estos entornos, decenas de miles de usuarios acceden simultáneamente al abrirse la boletería, compitiendo por un número finito de asientos con la expectativa de una experiencia interactiva en tiempo real y garantías absolutas de consistencia que eviten el *overbooking* (sobreventa).

Para este proyecto integrador, los estudiantes construirán un Sistema Distribuido de Reserva de Boletas denominado **Icesi Live Arena**, articulando dos grandes paradigmas:

1. **Backend de Alta Disponibilidad en Java (ZeroC Ice):** Expone un servicio RPC (*Remote Procedure Call*) multiprotocolo que atiende solicitudes concurrentes tanto en canal binario TCP (para clientes nativos) como en canal WebSocket (para clientes web).
2. **Frontend Modular en JavaScript o TypeScript Vainilla con Webpack:** Una Single Page Application (SPA) construida enteramente en JavaScript o TypeScript moderno (ES6+), estructurada mediante Componentes Reutilizables y Módulos, sin frameworks (sin React, Vue ni Angular). Dicha aplicación se empaquetará profesionalmente mediante Webpack.

Capa de Software	Tecnología y Protocolo	Responsabilidad en el Sistema
Frontend Web	Vanilla JS (ES Modules) + Webpack	Plano interactivo de sillas, carrito reactivo, temporizadores y modal de checkout.
Canal WebSocket	`ws://localhost:10002` (ZeroC Ice)	Conexión RPC bidireccional desde el navegador hacia el servidor.
Canal TCP	`default -p 10000` (ZeroC Ice)	Canal para clientes nativos y batería de pruebas automatizadas.
Servidor de Lógica	Java 17 + ZeroC Ice 3.7	Gestión del aforo (90 sillas), bloqueo temporal y confirmación de compra.

2 Especificación del Backend y Servicios RPC

2.1 Servicios Implementados en el Backend (Contrato Slice)

El servidor expone la interfaz remota `BookingService` definida en el archivo `BookingService.ice`. Esta interfaz encapsula las operaciones fundamentales para la consulta del recinto, reserva temporal y confirmación transaccional:

Operación RPC	Parámetros de Entrada	Retorno	Descripción y Excepciones
`getEventInfo()`	*Ninguno*	`EventInfo`	Retorna la metadata del concierto (nombre, fecha, aforo total de 90 sillas, precios por zona y contadores de sillas disponibles, bloqueadas y vendidas).
`getAllSeats()`	*Ninguno*	`SeatList` (`Seat[]`)	Retorna la lista completa de las 90 sillas con su identificación, zona, fila, número, precio, estado y cliente retenedor.
`getSeatsByZone(zone)`	`SeatZone zone`	`SeatList` (`Seat[]`)	Retorna las sillas filtradas por zona: `ZONEVIP`, `ZONEPLATEA` o `ZONEGENERAL`.
`getSeat(seatId)`	`string seatId`	`Seat`	Consulta el detalle de una silla específica por código (ej. `"A-1"`). Lanza `SeatNotFoundException` si no existe.
`lockSeat(seatId, clientId)`	`string seatId` `string clientId`	`LockResult`	Bloquea temporalmente una silla disponible por 90 segundos para el cliente indicado. Si el mismo cliente reintenta, renueva el tiempo. Lanza `SeatNotFoundException` o `SeatNotAvailableException`.
`releaseSeat(seatId, clientId)`	`string seatId` `string clientId`	`bool`	Libera voluntariamente el bloqueo temporal de una silla antes de que expire su tiempo. Retorna `true` si la operación fue exitosa. Lanza `SeatNotFoundException`.

`confirmBooking(seatIds, clientId, name, email)`	`StringSeq seatIds`
	`string clientId`
	`string customerName`
	`string customerEmail`
		`BookingResult`	Confirma la compra definitiva de los asientos bloqueados, cambia su estado a `BOOKED` y genera un código de reserva (`RES-XXXXXX`). Lanza `InvalidBookingException` o `BookingException`.

`getMyLockedSeats(clientId)`	`string clientId`	`SeatList`	Retorna las sillas actualmente retenidas por el cliente cuyo temporizador se encuentre activo.
`getMyBookings(clientId)`	`string clientId`	`BookingList`	Retorna el historial de compras confirmadas realizadas por el cliente con sus respectivos códigos y totales.
`resetSeats()`	*Ninguno*	`void`	Restablece el inventario a estado inicial (todas disponibles). Utilidad para pruebas de laboratorio.

2.2 Ciclo de Vida de los Asientos y Política de Bloqueo

Cada una de las 90 sillas del auditorio (20 VIP, 30 Platea y 40 General) transita por tres estados:

- **AVAILABLE (Disponible):** La silla está libre para cualquier cliente.
- **LOCKED (Bloqueo Temporal de 90 Segundos):** Al seleccionarse, el cliente adquiere un derecho de compra exclusivo temporal. Si el tiempo expira sin confirmarse el pago, la silla se libera automáticamente para otros usuarios.
- **BOOKED (Venta Definitiva):** Una vez confirmado el pago, la silla queda asignada de forma irreversible con su respectivo código de reserva alfanumérico (`RES-XXXXXX`).

3 Requerimientos del Sistema (Funcionales y No Funcionales)

3.1 Requerimientos Funcionales (RF)

RF-1: Visualización de Aforo y Precios en Tiempo Real: El sistema debe consultar y presentar la información del evento, precios por zona y el estado en tiempo real de las 90 sillas del recinto (Disponibles, Bloqueadas y Vendidas).

RF-2: Selección y Bloqueo Temporal Exclusivo: Al seleccionar un asiento disponible, el sistema debe solicitar su retención temporal exclusiva por 90 segundos al backend RPC. Dos usuarios no pueden bloquear la misma silla simultáneamente.

RF-3: Temporizador Reactivo y Desbloqueo Automático: La interfaz debe presentar una cuenta regresiva clara del tiempo restante de reserva. Al expirar los 90 segundos sin confirmación de compra, el asiento debe liberarse automáticamente en el servidor y quedar disponible para otros clientes.

RF-4: Gestión de Carrito y Liberación Manual: El usuario debe poder revisar las sillas en su carrito y removerlas voluntariamente antes de la expiración, liberándolas de inmediato en el backend mediante `releaseSeat`.

RF-5: Checkout y Confirmación de Compra: El usuario debe ingresar sus datos de contacto (nombre y correo electrónico válido) para confirmar la compra de sus sillas retenidas. El sistema debe validar que los bloqueos continúen vigentes, cambiar el estado a `BOOKED` y generar un código de reserva alfanumérico único (`RES-XXXXXX`).

RF-6: Consulta de Reservas del Cliente: El cliente debe poder consultar las compras confirmadas asociadas a su identificador de sesión.

RF-7: Manejo Resiliente de Excepciones Remotas: La interfaz debe capturar y notificar adecuadamente errores RPC del backend (sillas no disponibles, bloqueos expirados, desconexión del servidor) sin degradar la aplicación ni perder el estado local consistente.

3.2 Requerimientos No Funcionales y Restricciones Técnicas (RNF)

RNF-1: Consistencia e Integridad Concurrente (Thread-Safety): El backend debe garantizar exclusión mutua estricta en las operaciones críticas de inventario. Bajo ninguna condición de estrés concurrente se debe permitir el doble bloqueo o sobreventa de una misma silla.

RNF-2: Arquitectura Frontend en Vanilla JS (ES6+ / TypeScript): El frontend debe implementarse exclusivamente en JavaScript moderno (ES6+) o TypeScript sin frameworks (prohibido el uso de React, Angular, Vue o similares). El código debe organizarse en componentes desacoplados con responsabilidades únicas.

RNF-3: Tooling y Empaquetado Profesional con Webpack: El ciclo de vida del frontend debe administrarse con `npm` y empaquetarse con Webpack, proveyendo scripts `npm start` (desarrollo en puerto 3000) y `npm run build` (bundle optimizado para producción en `dist/`).

RNF-4: Interoperabilidad RPC Multi-Protocolo: El cliente web debe comunicarse con el backend exclusivamente mediante el canal WebSocket (`ws://localhost:10002`) empleando la librería oficial de ZeroC Ice para JavaScript, mientras que el canal TCP nativo (`10000`) se reserva para pruebas automatizadas.

4 Especificación del Frontend en JavaScript Vainilla con Webpack

4.1 Tooling y Empaquetado con Webpack

El frontend debe gestionarse mediante `npm` e incluir los siguientes elementos:

1. **Gestor de paquetes:** Archivo `package.json` estructurado con dependencias de desarrollo (`webpack`, `webpack-cli`, `webpack-dev-server`, loaders de CSS y HTML) y dependencias de ejecución (`ice`).

2. **Configuración Webpack (`webpack.config.js`):** Configuración que soporte ES Modules, bundle unificado, inyección automática de HTML y servidor de desarrollo en puerto `3000`.

3. **Scripts de npm:** Comandos `npm start` (entorno de desarrollo) y `npm run build` (generación de la distribución de producción en `dist/`).

4.2 Guía de Interfaz (Wireframe de Referencia)

La Figura 1 presenta un *wireframe* conceptual de referencia para el flujo de usuario en la aplicación. Este esquema ilustra las interacciones esenciales contempladas en los requerimientos del sistema: la visualización general del evento y aforo en tiempo real, el plano interactivo de selección de asientos con sus diferentes estados de ocupación, el panel lateral con la cuenta regresiva del bloqueo temporal y el formulario modal para la confirmación de la compra con su respectivo comprobante.

Libertad de diseño y propuesta de componentes: El prototipo suministrado tiene fines exclusivamente orientativos. Los estudiantes cuentan con total libertad para diseñar y proponer disposiciones visuales, jerarquías de elementos o estilos alternativos, siempre y cuando satisfagan a cabalidad los requerimientos funcionales y no funcionales definidos en la Sección 3. Es responsabilidad del equipo de estudiantes realizar el análisis de interfaz correspondiente y definir autónomamente la descomposición en componentes modulares y desacoplados dentro de su implementación en JavaScript o TypeScript vainilla.

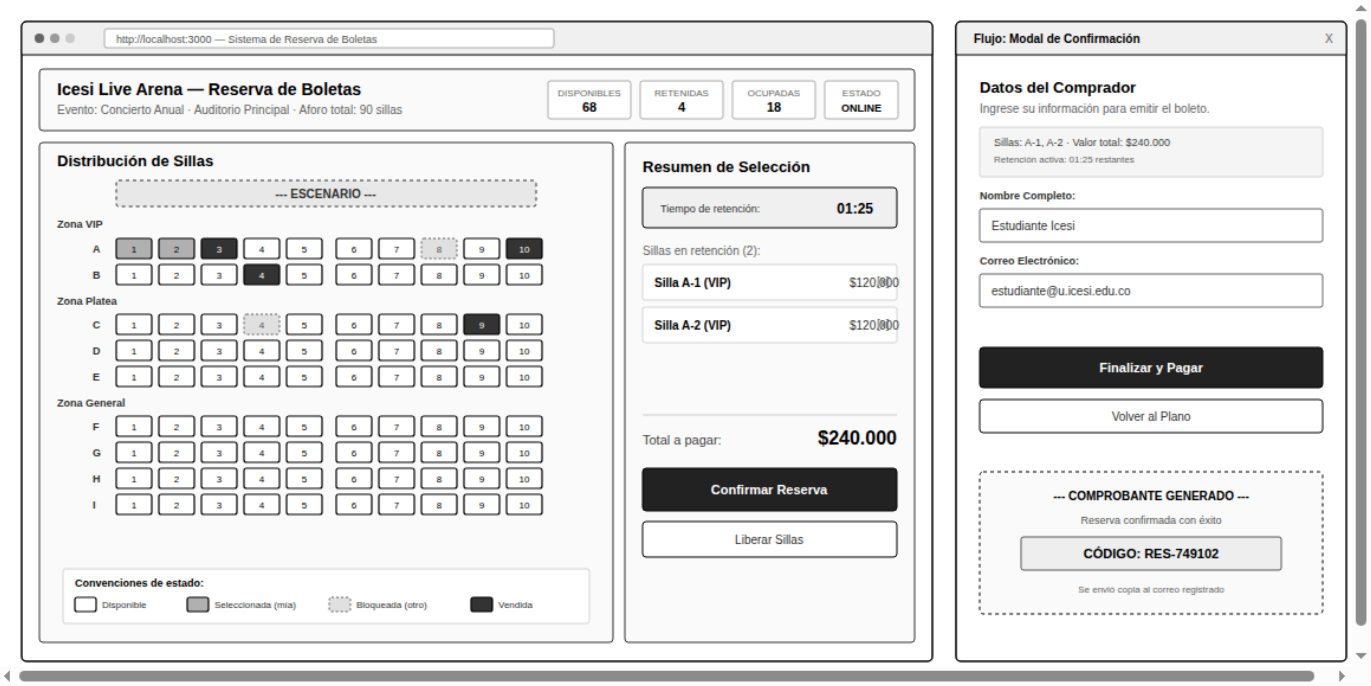


Figura 1: Wireframe de referencia para la interfaz de reserva de boletas y flujo de confirmación.

5 Procedimiento de Desarrollo y Tareas por Fases

El proyecto está diseñado para desarrollarse de manera progresiva:

5.1 Fase 1: Estructura del Frontend y Bundling con Webpack (Sesión 1)

1. Crear la carpeta `client-js/` e inicializar el proyecto con `npm init -y`.
2. Instalar Webpack y los loaders necesarios (`html-webpack-plugin`, `style-loader`, `css-loader`, `ice`).
3. Configurar `webpack.config.js` y validar la ejecución del entorno de desarrollo con `npm start` en el puerto `3000`.
4. Diseñar y desarrollar la estructura modular de la interfaz en JavaScript o TypeScript con estilos desacoplados, implementando los módulos y componentes requeridos para el plano de asientos, temporizador reactivo y confirmación de compra.

5.2 Fase 2: Integración RPC con ZeroC Ice vía WebSocket y Validación E2E (Sesión 2)

1. Configurar el cliente Webpack para inicializar el Communicator de ZeroC Ice y conectarse al endpoint WebSocket en `ws://localhost:10002`.
2. Vincular los eventos de clic del plano con las llamadas remotas `lockSeat` y `releaseSeat`.
3. Implementar la cuenta regresiva reactiva de 90 segundos con liberación automática ante expiración.
4. Conectar el flujo de checkout con `confirmBooking` y desplegar el comprobante con el código de reserva.
5. Realizar pruebas de usuario concurrentes abriendo múltiples ventanas de navegador en paralelo para comprobar la consistencia visual y remota del inventario.

6 Criterios de Evaluación y Sustentación Oral

6.1 Criterios de Evaluación Técnica del Proyecto (Nota Base)

La solución técnica entregada se evaluará con base en los siguientes criterios sobre una escala base de **0.0 a 5.0 puntos**:

Criterio de Evaluación	Porcentaje
Tooling y Webpack	20 %
Arquitectura Frontend Vanilla JS	20 %
Integración RPC ZeroC Ice	20 %
Concurrencia y Thread-Safety	20 %
UX, Temporizador y Compra	20 %
Despliegue en una VM de la sala	10 % adicional
Total	**100 %**

El despliegue aplica para quienes hicieron el despliegue el 8 de noviembre o lo realicen y sustenten para esta entrega.

6.2 Sustentación Oral Individual (Factor Multiplicativo)

La sustentación oral es de carácter estrictamente individual y se aplica como un **factor multiplicativo** ($F_s \in [0{,}0,\ 1{,}0]$) sobre la nota base obtenida por el proyecto en la evaluación técnica:

$$\text{Calificación Definitiva Individual} = \text{Nota Base del Proyecto} \times F_s$$

El factor de sustentación (F_s) evalúa la solvencia conceptual, la autoría del código y la argumentación de las decisiones de diseño mediante preguntas técnicas y posibles modificaciones en vivo formuladas por el docente:

- $F_s = 1{,}0$ **(100 %):** Dominio técnico sobresaliente de la arquitectura global (backend ZeroC Ice, empaquetado con Webpack, sincronización de hilos y manipulación del DOM). Justifica con claridad todas las decisiones de diseño y responde con solidez a preguntas teóricas o modificaciones en vivo.
- $F_s = 0{,}7 - 0{,}9$ **(70 % – 90 %):** Buen dominio general del funcionamiento del sistema, aunque con dudas leves o imprecisiones al explicar detalles técnicos puntuales de la implementación.
- $F_s = 0{,}4 - 0{,}6$ **(40 % – 60 %):** Dominio técnico limitado o parcial. Dificultad para justificar segmentos clave del código entregado o evidencia de dependencia excesiva en otros integrantes del equipo.
- $F_s = 0{,}0 - 0{,}3$ **(0 % – 30 %):** Desconocimiento sustancial del código fuente, incapacidad de explicar los flujos básicos de la aplicación o inasistencia no justificada a la sustentación oral.

7 Instrucciones y Políticas de Entrega

1. **Repositorio de Código:** La entrega se realiza a través del repositorio Git enlazado en la organización del curso. La raíz del proyecto debe incluir:
   - `codigo_base/`: Con el servidor Java y la vulnerabilidad de concurrencia debidamente corregida y probada.
   - `client-js/`: Con el proyecto frontend completo, configuración de Webpack, componentes y estilos.
2. **Historial de Commits:** Se exige un historial continuo con mensajes descriptivos bajo el estándar *Conventional Commits* (`feat`, `fix`, `refactor`).
3. **Video Demostrativo (Screencast):** Enlace a un video público o institucional de máximo 6 minutos demostrando:
   - La ejecución del servidor Java y la prueba de estrés de concurrencia corregida.
   - La compilación y ejecución de la aplicación web con `npm start` mediante Webpack.
   - El flujo completo de navegación, bloqueo de sillas, cuenta regresiva del temporizador y emisión de la reserva.
4. **Sustentación Oral:** Cada estudiante presentará y defenderá su proyecto de forma presencial o sincrónica en el laboratorio en la fecha estipulada, donde se evaluará individualmente el factor multiplicativo F_s.